

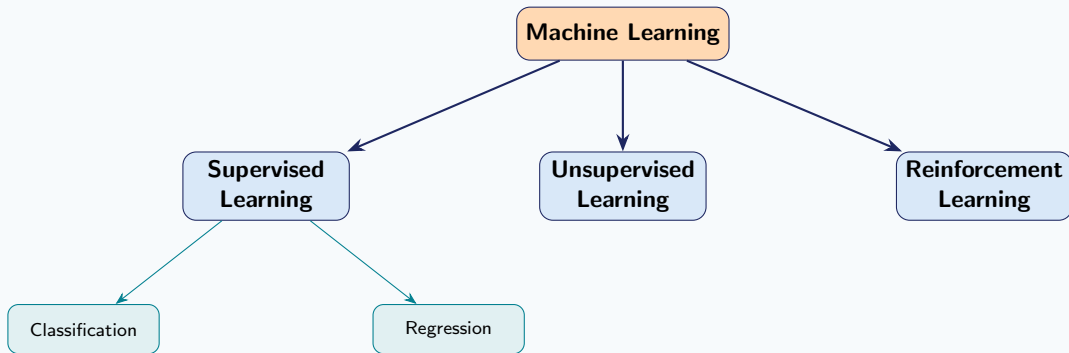
Classification Techniques in Machine Learning

Dr. Shailesh Sivan

- 1 Introduction
- 2 Logistic Regression
- 3 Support Vector Machine
- 4 K-Nearest Neighbour
- 5 Naïve Bayes
- 6 Decision Trees
- 7 Evaluation Metrics
- 8 Binary Classification Metrics
- 9 Multiclass Classification Metrics
- 10 Choosing the Right Metric
- 11 Worked Examples
- 12 Practice Problems

What is Machine Learning?

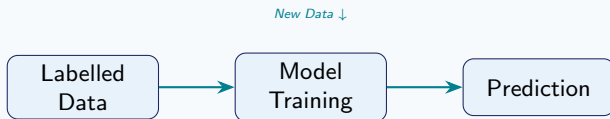
Machine learning enables systems to **learn from data** and improve without being explicitly programmed.



Definition

A model **learns from past labelled data** and makes predictions on new, unseen inputs.

Pipeline:



Types of Supervised Learning:

- **Classification** — predict a discrete label
- **Regression** — predict a continuous value

Covered in this lecture:

- 1 Logistic Regression
- 2 Support Vector Machine (SVM)
- 3 K-Nearest Neighbour (KNN)
- 4 Naïve Bayes
- 5 Decision Trees

What is Logistic Regression?

Definition

Logistic Regression is a **supervised classification** algorithm that models the **probability** that an input $\mathbf{x}$ belongs to a given class using the **sigmoid function**.

Key : instead of predicting a value directly, predict

$$P(y = 1 | \mathbf{x}) = \sigma(z), \quad z = \mathbf{w}^\top \mathbf{x} + b$$

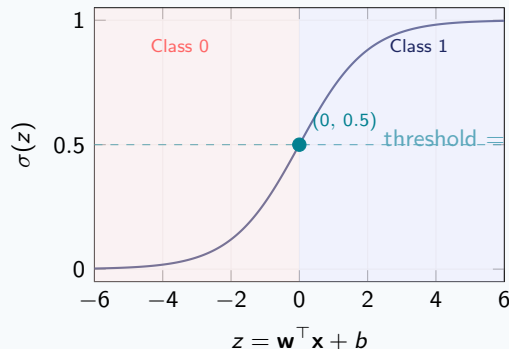
Sigmoid (Logistic) Function

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Decision rule:

$$\hat{y} = \begin{cases} 1 & \sigma(z) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Sigmoid Function



$$\sigma(z) \rightarrow 0 \text{ as } z \rightarrow -\infty; \quad \sigma(z) \rightarrow 1 \text{ as } z \rightarrow +\infty$$

Log-Loss (Binary Cross-Entropy)

$$J(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

where $\hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$.

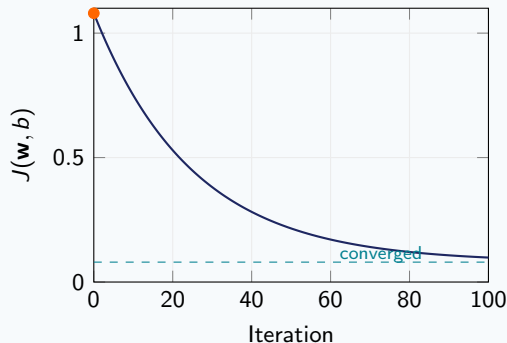
Why not MSE? Log-loss is **convex** — guaranteed global minimum.

Gradient Descent Update

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}}, \quad b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i) \mathbf{x}_i$$

Log-Loss Convergence



Regularisation (prevent overfitting):

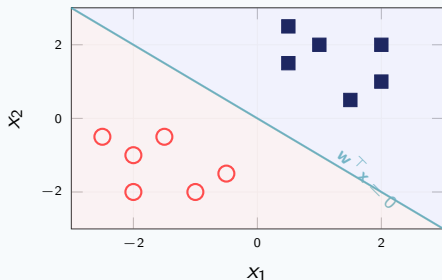
$$J_{\text{reg}} = J + \frac{\lambda}{2n} \|\mathbf{w}\|^2 \quad (\text{L2 / Ridge})$$

Decision Boundary, Multi-class & Key Properties

Linear Decision Boundary: The boundary is where $\sigma(z) = 0.5$, i.e. $z = 0$:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

Decision Boundary



Multi-class: Softmax Regression

For $K > 2$ classes, generalise to **softmax**:

$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^T \mathbf{x} + b_k}}{\sum_{j=1}^K e^{\mathbf{w}_j^T \mathbf{x} + b_j}}$$

One-vs-Rest (OvR): train K binary classifiers.

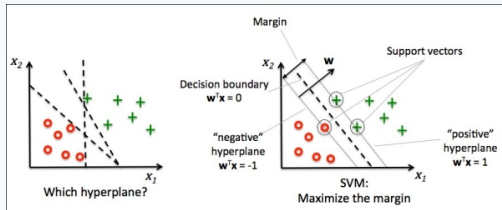
Key Properties

- **Output:** probability $\in (0, 1)$ — well-calibrated
- **Boundary:** linear in feature space
- **Convex loss** — efficient, guaranteed convergence
- **Sensitive** to outliers and feature , normalise inputs
- **Regularise** with L1 (sparse) or L2 (small weights)

Definition

An **SVM** is a supervised ML algorithm for classification or regression. It finds the **hyperplane** that best separates the classes with **maximum margin**.

SVM Workflow:



Key Concepts:

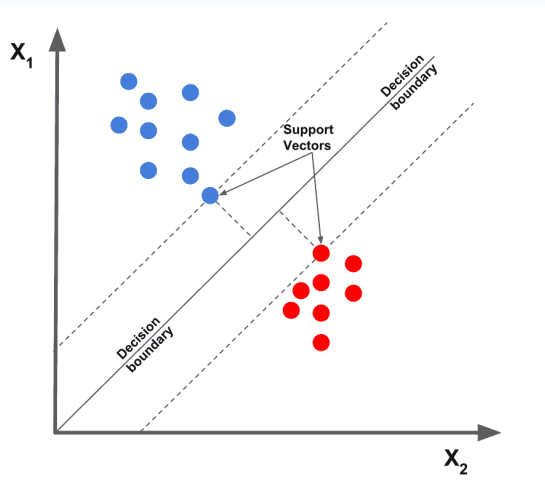
- **Hyperplane** — decision boundary
- **Support Vectors** — nearest data points to the boundary
- **Margin** — distance from boundary to support vectors
- Goal: **maximise the margin**

Height-Weight data used to illustrate classification:

Female	
Height (cm)	Weight (kg)
174	65
174	88
175	75
180	65
185	80

Male	
Height (cm)	Weight (kg)
179	90
180	80
183	80
187	85
182	72

Goal: find a [hyperplane](#) that separates Female from Male data points.

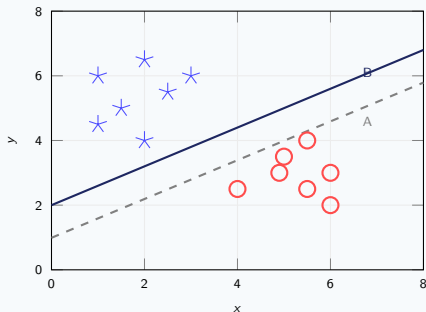


Optimal Hyperplane

The hyperplane that **maximises the margin** between the two classes.

- Dashed lines = **margin boundaries**
- Points on dashed lines = **support vectors**
- The distance between support vectors and hyperplane should be **as large as possible**

Scenario 3: Hyperplane A vs B



Scenario 3

Both A and B separate the classes correctly. **B has a higher margin** → B is the **correct choice**.

Scenario 4: Outlier

When an outlier exists from one class, SVM uses a **soft-margin** formulation to ignore it.

Scenario 5: Non-linear

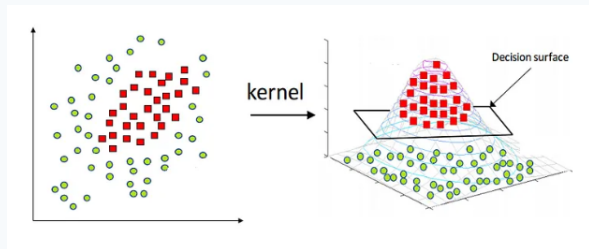
Classes not linearly separable → use the **kernel trick** to add features.

Problem: Red circles and blue stars cannot be separated by a line in (x, y) .

Solution: Introduce a new feature

$$z = x^2 + y^2$$

- $z \geq 0$ always (squared sum)
- Red circles: close to origin $\Rightarrow$ low z
- Blue stars: away from origin $\Rightarrow$ high z



Projecting to (x, z) makes the classes **linearly separable**.

Intuition

"If it walks like a duck, quacks like a duck, then it's probably a duck."

k -NN Classification Rule

Assign to a test sample the **majority class label** of its k nearest training samples.

Process:

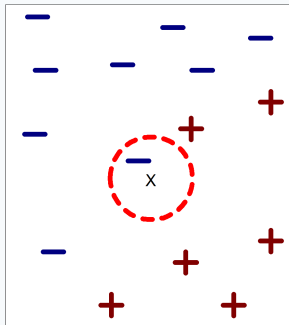


Key notes:

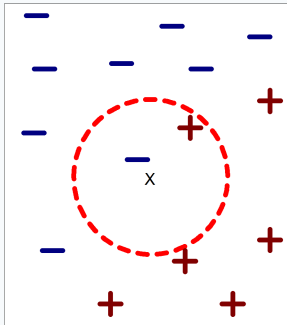
- k usually chosen **odd** (avoid ties)
- $k = 1$: nearest-neighbour rule
- **Rule of thumb:** $k \approx \sqrt{N}$

Definition

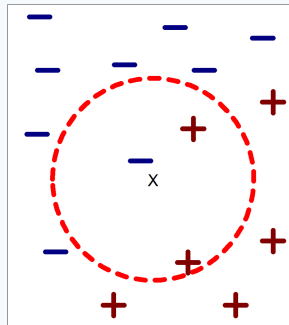
The k -nearest neighbours of a record x are the k data points in the training set with the **smallest distance** to x .



(a) 1-nearest neighbor



(b) 2-nearest neighbor



(c) 3-nearest neighbor

Basic k -NN (majority vote):

$$\hat{f}(q) = \arg \max_{v \in V} \sum_{i=1}^k \delta(v, f(x_i))$$

Distance-Weighted k -NN (closer $\Rightarrow$ more influence):

$$\hat{f}(q) = \arg \max_{v \in V} \sum_{i=1}^k \frac{1}{[d(x_i, x_q)]^2} \delta(v, f(x_i))$$

General kernel functions (e.g. *Parzen Windows*) may replace inverse distance.

Predicting Continuous Values: Replace majority vote with weighted average:

$$\hat{f}(q) = \frac{\sum_{i=1}^k w_i f(x_i)}{\sum_{i=1}^k w_i} \quad (\text{unweighted: } w_i = 1)$$

Issues in k -NN

- Choice of k
- Distance metric
- Computational complexity
- Size of training set
- Dimensionality of data

Choosing k

- k too **small** $\Rightarrow$ sensitive to noise
- k too **large** $\Rightarrow$ neighbourhood includes other classes

Rule of Thumb:

$$k \approx \sqrt{N}$$

where N = number of training points.

Metric	Formula
Minkowski	$D = (\sum x_i - y_i ^r)^{1/r}$
Euclidean	$D = \sqrt{\sum (x_i - y_i)^2}$
Manhattan	$D = \sum x_i - y_i $
Canberra	$D = \sum \frac{ x_i - y_i }{ x_i + y_i }$
Chebyshev	$D = \max x_i - y_i $
Quadratic	$D = (\mathbf{x} - \mathbf{y})^T Q (\mathbf{x} - \mathbf{y})$, Q : pos.-def. weight matrix
Mahalanobis	$D = [\det V]^{1/m} (\mathbf{x} - \mathbf{y})^T V^{-1} (\mathbf{x} - \mathbf{y})$
Correlation	$D = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$
Chi-square	$D = \sum \frac{1}{\text{sum}_i} \left(\frac{x_i}{\text{size}_x} - \frac{y_i}{\text{size}_y} \right)^2$
Kendall	$D = 1 - \frac{2}{n(n-1)} \sum_{i>j} \text{sign}(x_i - x_j) \text{sign}(y_i - y_j)$

Posterior probability of class C given features $\mathbf{X}$:

$$P(C | \mathbf{X}) \propto P(\mathbf{X} | C) P(C) = P(X_1, \dots, X_n | C) P(C)$$

Difficulty

Learning the full joint probability $P(X_1, \dots, X_n | C)$ is **intractable** for high-dimensional data.

Naïve Bayes assumption — all attributes are **conditionally independent**:

$$P(X_1, \dots, X_n | C) = P(X_1 | C) P(X_2 | C) \cdots P(X_n | C)$$

MAP Classification Rule:

$$[P(x_1 | c^*) \cdots P(x_n | c^*)] P(c^*) > [P(x_1 | c) \cdots P(x_n | c)] P(c) \quad \forall c \neq c^*$$

Learning Phase

- 1: **for all** target $c_i \in \{c_1, \dots, c_L\}$ **do**
- 2: $\hat{P}(C=c_i) \leftarrow$ estimate from S
- 3: **for all** attr. value a_{jk} of x_j **do**
- 4: $\hat{P}(X_j=a_{jk}|C=c_i) \leftarrow$ est.
- 5: **end for**
- 6: **end for**

Output: conditional probability tables

Test Phase

Require: Instance $X' = (a'_1, \dots, a'_n)$

- 1: Look up tables
- 2: Assign c^* to X' if

$$\prod_i \hat{P}(a'_i|c^*) \hat{P}(c^*) > \prod_i \hat{P}(a'_i|c) \hat{P}(c)$$

for all $c \neq c^*$.

Naïve Bayes — Example: Play Tennis (Training Data)

Day	Outlook	Temp	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

$$P(\text{Play}=\text{Yes}) = \frac{9}{14}, \quad P(\text{Play}=\text{No}) = \frac{5}{14}$$

Naïve Bayes — Conditional Probability Tables

Outlook	Yes	No
Sunny	2/9	3/5
Overcast	4/9	0/5
Rain	3/9	2/5

Humidity	Yes	No
High	3/9	4/5
Normal	6/9	1/5

Temperature	Yes	No
Hot	2/9	2/5
Mild	4/9	2/5
Cool	3/9	1/5

Wind	Yes	No
Strong	3/9	3/5
Weak	6/9	2/5

New instance: $x' = (\text{Sunny}, \text{Cool}, \text{High}, \text{Strong})$

Play = Yes:

$$P(\text{Sunny}|\text{Yes}) = 2/9$$

$$P(\text{Cool}|\text{Yes}) = 3/9$$

$$P(\text{High}|\text{Yes}) = 3/9$$

$$P(\text{Strong}|\text{Yes}) = 3/9$$

$$P(\text{Yes}) = 9/14$$

$$P(\text{Yes}|x') \approx 0.0053$$

Play = No:

$$P(\text{Sunny}|\text{No}) = 3/5$$

$$P(\text{Cool}|\text{No}) = 1/5$$

$$P(\text{High}|\text{No}) = 4/5$$

$$P(\text{Strong}|\text{No}) = 3/5$$

$$P(\text{No}) = 5/14$$

$$P(\text{No}|x') \approx 0.0206$$

Decision

Since $P(\text{Yes}|x') < P(\text{No}|x')$, we label x' as **No** (Do not play tennis).

Topics covered:

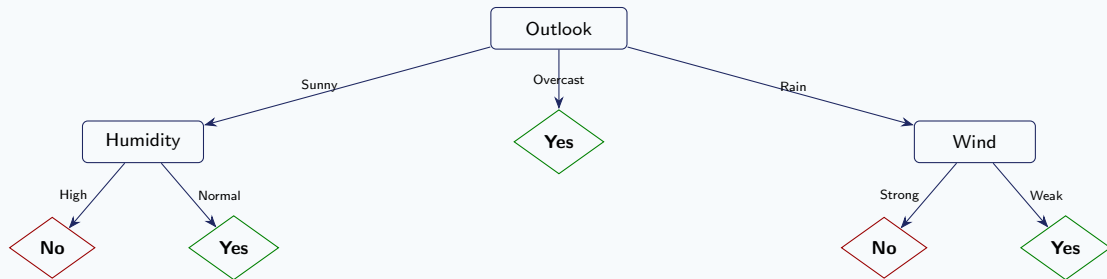
- 1 Decision Tree Representations
- 2 ID3 and C4.5 algorithms (Quinlan 1986)
- 3 CART algorithm (Breiman et al. 1985)
- 4 Entropy & Information Gain
- 5 Overfitting

Key idea

A tree of **if-then rules** that partitions the feature space into axis-parallel rectangles, each labelled with a class.

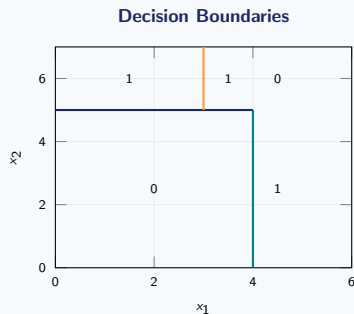
- **Internal nodes** — test a feature x_j
- **Branches** — feature values
- **Leaf nodes** — class label $h(\mathbf{x})$

Decision Trees — Example Tree (Play Tennis)

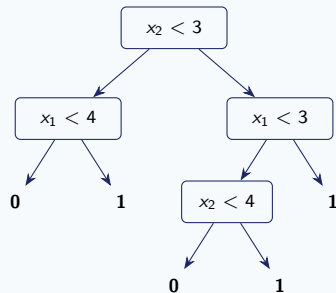


Example: $x = (\text{Sunny}, \text{Hot}, \text{High}, \text{Strong}) \rightarrow$ follows Outlook $\rightarrow$ Sunny $\rightarrow$ Humidity $\rightarrow$ High $\rightarrow$ No. Temperature is irrelevant.

Decision boundaries are **axis-parallel rectangles**:



Corresponding tree:



Decision Trees — Learning Algorithm (GrowTree)

Given $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$, $\mathbf{x} = (x_1, \dots, x_d)$, $x_j, y \in \{0, 1\}$:

GrowTree(S)

```
1: if  $y = 0$  for all  $\langle \mathbf{x}, y \rangle \in S$  then return new leaf(0)
2: else if  $y = 1$  for all  $\langle \mathbf{x}, y \rangle \in S$  then return new leaf(1)
3: else
4:   choose best attribute  $x_j$ 
5:    $S_0 \leftarrow$  all  $\langle \mathbf{x}, y \rangle \in S$  with  $x_j = 0$ 
6:    $S_1 \leftarrow$  all  $\langle \mathbf{x}, y \rangle \in S$  with  $x_j = 1$  return new node( $x_j$ , GROWTREE( $S_0$ ), GROWTREE( $S_1$ ))
7: end if
```

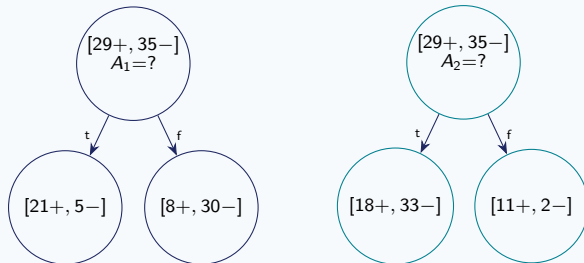
Open question

What happens if features are not binary? What about regression?

Decision Trees — Choosing the Best Attribute

Many frameworks exist. We use **Entropy Gain**.

Which split is better — A_1 or A_2 ?



Observation

A_1 produces **purer** child nodes ($[21+, 5-]$ and $[8+, 30-]$) than A_2 ($[18+, 33-]$ and $[11+, 2-]$).

$\Rightarrow A_1$ has **higher Information Gain**.

Decision Trees — Entropy

Let $p_{\oplus}$ = proportion of positive examples, $p_{\ominus}$ = proportion of negative examples in S .

Binary Entropy

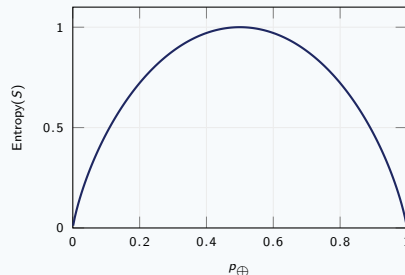
$$\text{Entropy}(S) \equiv -p_{\oplus} \log_2 p_{\oplus} - p_{\ominus} \log_2 p_{\ominus}$$

General Entropy (C classes)

$$H(S) = - \sum_{c \in C} p_c \log_2 p_c$$

- $H = 0$: set is **pure**
- $H = 1$: classes equally mixed

Entropy Curve



Dominant class $\Rightarrow$ low entropy
Equiprobable $\Rightarrow$ high entropy

Information Gain

$$\text{Gain}(S, A) \equiv \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

- $\text{Gain}(S, A)$ = expected **reduction in entropy** due to sorting on A
- S_v = subset of S where $A = v$
- Choose the attribute with the **highest** gain

Equivalently (*IG* notation)

$$IG(S, F) = H(S) - \sum_{f \in F} \frac{|S_f|}{|S|} H(S_f)$$

IG measures the **increase in certainty** about S once F is known.

Decision Trees — Computing IG : Example

Root node: $S = [9+, 5-]$, $H(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} \approx 0.940$

Feature: Wind

$$S_{\text{weak}} = [6+, 2-] \Rightarrow H = 0.811$$

$$S_{\text{strong}} = [3+, 3-] \Rightarrow H = 1.000$$

$$IG(S, \text{wind}) = 0.940 - \frac{8}{14}(0.811) - \frac{6}{14}(1.000) = 0.048$$

All information gains at root:

Attribute	IG
Outlook	0.246 ← maximum
Humidity	0.151
Wind	0.048
Temperature	0.029

Outlook chosen as the root node.

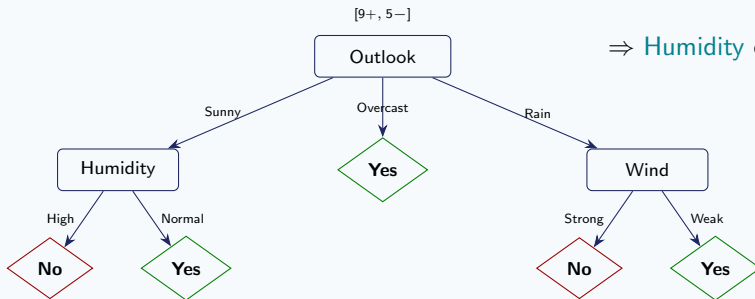
Sunny branch ($S_{\text{Sunny}} = \{D1, D2, D8, D9, D11\}$):

$$IG(\text{Humidity}) = 0.970$$

$$IG(\text{Temperature}) = 0.570$$

$$IG(\text{Wind}) = 0.019$$

$\Rightarrow$ **Humidity** chosen for Sunny branch.



Algorithm terminates when

All examples in a node have the **same class label**.

Support Vector Machine

- Finds max-margin hyperplane
- Kernel trick for non-linear data
- Effective in high dimensions

Naïve Bayes

- Conditional independence assumption
- Learns $P(X_j|C)$ from data
- MAP rule for prediction

K-Nearest Neighbour

- Non-parametric, lazy learner
- Distance-weighted voting
- Rule of thumb: $k \approx \sqrt{N}$

Decision Trees

- Axis-parallel decision boundaries
- ID3/C4.5: entropy gain criterion
- GROWTREE recursive algorithm

The Confusion Matrix (Binary)

		Predicted	
		Positive	Negative
Actual	Pos	TP	FN
	Neg	FP	TN

- **TP** — True Positive: correctly predicted positive
- **TN** — True Negative: correctly predicted negative
- **FP** — False Positive: negative predicted as positive
- **FN** — False Negative: positive predicted as negative

Example: Spam Detection

100 emails: 40 spam, 60 not spam

	Pred Spam	Pred Ham
Spam	35	5
Ham	8	52

TP=35, FN=5, FP=8, TN=52

Precision and Recall

Precision (Positive Predictive Value)

$$\text{Precision} = \frac{TP}{TP + FP}$$

"Of all predicted positives, how many are actually positive?"

Penalizes false alarms (FP)

Recall (Sensitivity / True Positive Rate)

$$\text{Recall} = \frac{TP}{TP + FN}$$

"Of all actual positives, how many did we catch?"

Penalizes missed cases (FN)

Spam Example (TP=35, FP=8, FN=5)

$$\text{Precision} = \frac{35}{35 + 8} \approx \mathbf{81.4\%}$$

$$\text{Recall} = \frac{35}{35 + 5} = \mathbf{87.5\%}$$

The Precision-Recall Trade-off

- High threshold $\Rightarrow$ High Precision, Low Recall
- Low threshold $\Rightarrow$ Low Precision, High Recall
- **Cancer detection:** prioritize Recall
- **Spam filter:** prioritize Precision

F1 Score, F_β Score & Specificity

F1 Score — Harmonic Mean of P & R

$$F_1 = \frac{2 TP}{2 TP + FP + FN}$$

Balances Precision and Recall; useful for imbalanced data. Range: $[0, 1]$.

F_β Score (Generalized)

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Prec} \times \text{Rec}}{\beta^2 \cdot \text{Prec} + \text{Rec}}$$

$\beta > 1$: emphasizes Recall; $\beta < 1$: emphasizes Precision
E.g., F_2 for disease detection, $F_{0.5}$ for spam.

Specificity (True Negative Rate)

$$\text{Specificity} = \frac{TN}{TN + FP}$$

“Of all negatives, how many were correctly identified?”

Spam Example

TP=35, FP=8, FN=5, TN=52

$$F_1 = 2 \cdot \frac{0.814 \times 0.875}{0.814 + 0.875} \approx \mathbf{84.3\%}$$

$$\text{Specificity} = \frac{52}{52 + 8} \approx \mathbf{86.7\%}$$

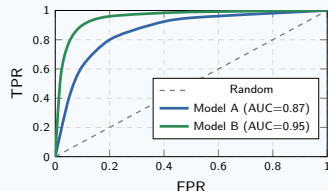
Key Insight

A well-calibrated model achieves high **Recall** (catches positives) and high **Specificity** (avoids false alarms). These two quantities capture complementary aspects of classifier quality.

ROC Curve & AUC

ROC = Receiver Operating Characteristic. Plots **TPR** vs **FPR** at every decision threshold.

$$\text{TPR} = \frac{TP}{TP+FN} \quad \text{FPR} = \frac{FP}{FP+TN}$$



AUC — Area Under ROC Curve

- **AUC = 1.0**: Perfect classifier
- **AUC = 0.5**: Random guessing
- **AUC < 0.5**: Worse than random

Interpretation

AUC = probability a random **positive** is ranked higher than a random **negative** by the model.

When to use AUC

- Comparing classifiers at all thresholds
- Threshold is not yet fixed
- Classes moderately balanced

Extending to Multiclass: The Confusion Matrix

For K classes, the matrix is $K \times K$:

	Cat	Dog	Bird
Cat	50	5	2
Dog	3	45	4
Bird	1	6	40

Per-class metrics (one-vs-rest):

$$\text{Prec}_i = \frac{C_{ii}}{\sum_j C_{ji}}, \quad \text{Rec}_i = \frac{C_{ii}}{\sum_j C_{ij}}$$

Macro Averaging

Average **equally** across classes:

$$\text{Macro-F1} = \frac{1}{K} \sum_{i=1}^K F1_i$$

Best when all classes are equally important.

Weighted Averaging

Weighted by **class support**: $\text{W-F1} = \frac{\sum_i n_i \cdot F1_i}{\sum_i n_i}$

Good for imbalanced multiclass problems.

Micro Averaging

Aggregate TP, FP, FN **globally**:

$$\text{Micro-F1} = \frac{2 \sum TP_i}{2 \sum TP_i + \sum FP_i + \sum FN_i}$$

Dominated by largest class.

Macro vs Micro vs Weighted — When to Use What

Strategy	Best Used When	Watch Out For
Macro	All classes equally important; detecting rare classes matters	Dominated by small classes
Micro	Large datasets; overall performance	Dominated by large classes
Weighted	Imbalanced classes; need a fair single metric	Hides poor minority performance

Example (Animal Classifier)

Classes: Cat (57), Dog (52), Bird (47)

$F1_{\text{Cat}} = 0.89$, $F1_{\text{Dog}} = 0.84$, $F1_{\text{Bird}} = 0.82$

Macro-F1 = $\frac{0.89+0.84+0.82}{3} = 0.85$

Key Takeaway

Medical diagnosis with rare classes $\Rightarrow$ **Macro**

Product categorization with millions of examples
 $\Rightarrow$ **Micro or Weighted**

Cohen's Kappa & Matthews Correlation Coefficient

Cohen's Kappa (κ)

Measures agreement adjusted for chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

p_o = observed accuracy, p_e = expected by chance

κ	Interpretation
< 0	Less than chance
0.0–0.2	Slight
0.2–0.4	Fair
0.4–0.6	Moderate
0.6–0.8	Substantial
0.8–1.0	Almost perfect

Matthews Correlation Coefficient (MCC)

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

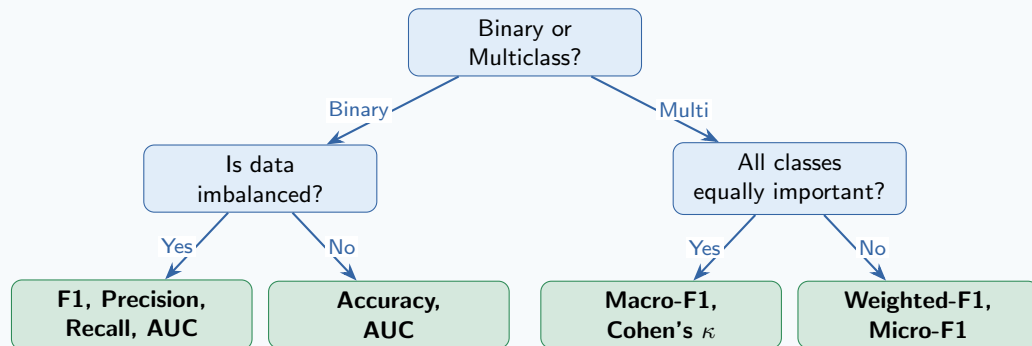
- Range: $[-1, +1]$
- +1: Perfect; 0: Random; -1: Inverse
- Works well for imbalanced classes
- One of the most informative single metrics

MCC Example

TP=35, TN=52, FP=8, FN=5

$$\text{MCC} = \frac{35 \cdot 52 - 8 \cdot 5}{\sqrt{43 \cdot 40 \cdot 60 \cdot 57}} \approx \mathbf{0.73}$$

Decision Framework: Choosing the Right Metric



Always examine the **full confusion matrix** — no single metric tells the whole story.

Worked Example 1: Medical Diagnosis

Problem Setup

HIV Test: 200 patients — 30 HIV+, 170 HIV−. Model predicts: 25 HIV+, 175 HIV−

Confusion Matrix:

	Pred+	Pred−
Act+	22	8
Act−	3	167

Metrics:

$$\text{Accuracy} = \frac{22 + 167}{200} = 94.5\%$$

$$\text{Precision} = \frac{22}{22 + 3} = 88.0\%$$

$$\text{Recall} = \frac{22}{22 + 8} = 73.3\%$$

$$F_1 = \frac{2 \times 0.88 \times 0.733}{0.88 + 0.733} \approx 80.0\%$$

Analysis

Recall = **73.3%**: 8 HIV+ patients missed. Dangerous in medicine. **Lower the threshold** to catch more positives (increases FP, reduces FN).

Worked Example 2: Sentiment Analysis (Multiclass)

Problem: 3-class sentiment — Positive, Neutral, Negative (300 samples each)

Per-class results:

Class	Prec	Rec	F1
Positive	0.82	0.88	0.85
Neutral	0.74	0.70	0.72
Negative	0.90	0.85	0.87
Macro	0.82	0.81	0.81
Weighted	0.82	0.81	0.81

Observations:

- Neutral class has lowest F1 (0.72) — harder to classify
- Macro = Weighted here (balanced 300 each)
- When classes are **unbalanced**, Macro $\neq$ Weighted

Action: Investigate Neutral class. Consider:

- More training data for Neutral
- Better features for ambiguous sentiment
- Per-class threshold tuning

Practice Problem 1

Problem

A fraud detection model is tested on 1000 transactions: 50 fraudulent, 950 legitimate.

	Pred Fraud	Pred Legit
Fraud	40	10
Legit	20	930

Compute: **(a)** Accuracy **(b)** Precision **(c)** Recall **(d)** F1 **(e)** Specificity

Discussion Question

The bank wants to **minimize fraud losses** (missing fraud is costly). Which metric should they optimize, and should they raise or lower the classification threshold?

Practice Problem 1 — Solution

TP = 40, FP = 20, FN = 10, TN = 930

$$(a) \text{ Accuracy} = \frac{40 + 930}{1000} = \mathbf{97.0\%}$$

$$(b) \text{ Precision} = \frac{40}{40 + 20} = \mathbf{66.7\%}$$

$$(c) \text{ Recall} = \frac{40}{40 + 10} = \mathbf{80.0\%}$$

$$(d) F_1 = \frac{2 \times 0.667 \times 0.8}{0.667 + 0.8} \approx \mathbf{72.7\%}$$

$$(e) \text{ Specificity} = \frac{930}{930 + 20} = \mathbf{97.9\%}$$

Discussion Answer

- Optimize **Recall** — minimize missed fraud (FN)
- **Lower** the threshold $\Rightarrow$ classify more as fraud $\Rightarrow$ fewer missed cases
- Trade-off: more false alarms (FP), lower Precision
- Use F_2 **score** ($\beta = 2$) to weight Recall twice as much as Precision

Practice Problem 2

Problem: 4-Class Image Classifier

A model classifies 400 images across 4 classes (100 each):

	Cat	Dog	Car	Tree
Cat	85	8	2	5
Dog	6	80	4	10
Car	1	2	90	7
Tree	3	5	2	90

- (a) Compute per-class Precision and Recall for “Dog.”
- (b) Compute overall Accuracy.
- (c) Which class performs worst? Why?
- (d) Would you report Macro-F1 or Weighted-F1 here, and why?

(a) Dog class:

$$\text{Prec}_{\text{Dog}} = \frac{80}{8 + 80 + 2 + 5} = \frac{80}{95} \approx \mathbf{84.2\%}$$

$$\text{Rec}_{\text{Dog}} = \frac{80}{6 + 80 + 4 + 10} = \frac{80}{100} = \mathbf{80.0\%}$$

(b) Overall Accuracy:

$$\text{Acc} = \frac{85 + 80 + 90 + 90}{400} = \frac{345}{400} = \mathbf{86.25\%}$$

(c) Worst class:

Dog has the most **false negatives** (20 missed) — particularly confused with Tree (10 cases) and Cat (6 cases).

(d) Averaging strategy:

Since all classes have **equal support** (100 each),
Macro-F1 = Weighted-F1.

To emphasize rare class performance, report
Macro-F1.

Insight

Even with 86% accuracy, Dog needs improvement — the confusion matrix reveals **where** errors occur.

Summary & Key Takeaways

Binary Classification

- **Confusion Matrix** — foundation of all metrics
- **Precision** — quality of predicted positives
- **Recall** — coverage of actual positives
- **F1** — harmonic mean of P & R
- **AUC-ROC** — threshold-free discrimination
- **MCC** — robust even with class imbalance

Multiclass Classification

- Extend via **one-vs-rest** per class
- **Macro** — equal weight per class
- **Weighted** — proportional to class size
- **Micro** — aggregate all errors globally
- **Cohen's κ** — beyond-chance agreement

Golden Rules

- 1 **Never** use accuracy alone on imbalanced data
- 2 Always **inspect** the full confusion matrix
- 3 Match metric to the **cost of each error type**
- 4 **No single number** captures full performance
- 5 Always report **multiple metrics** together

Key Formulas

$$F_1 = \frac{2 TP}{2 TP + FP + FN}$$
$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$